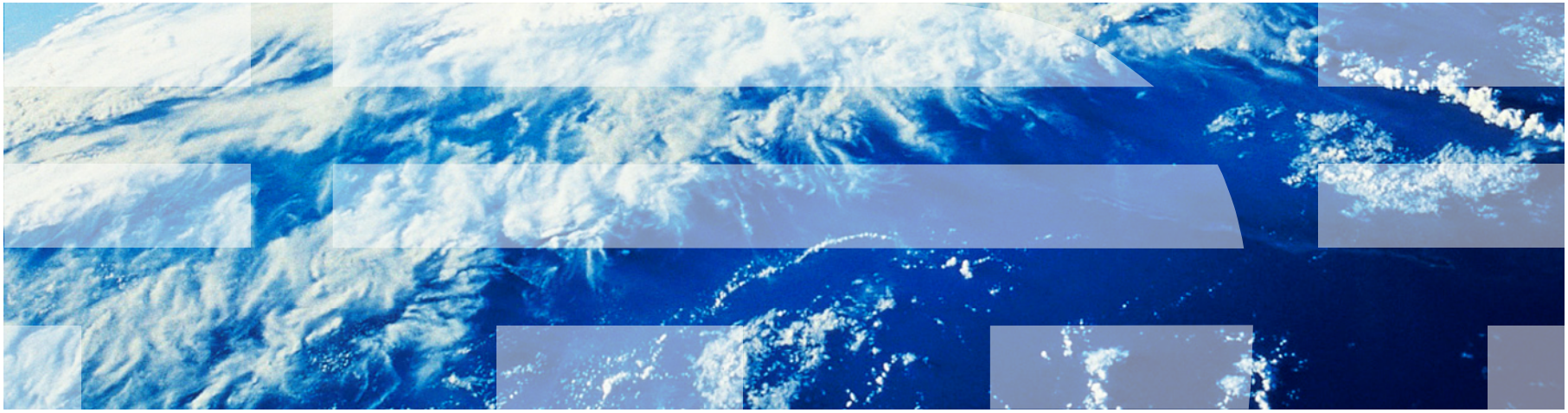
A Note: ACID is contentious!

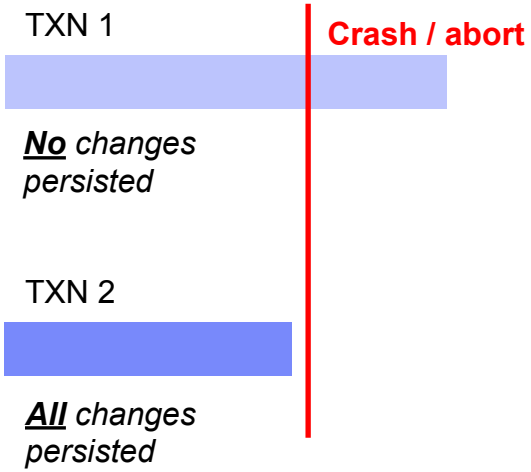
- Many debates over ACID, both **historically** and **currently**
- Many SQL databases do not provide ACID by default
  - Often provide read-committed transactions, a weaker form of isolation
- “NoSQL” DBs relax ACID even more
- In turn, now “NewSQL” reintroduces ACID compliance to NoSQL-style DBs…

Atomicity and Durability via Logging



Goal: Ensuring Atomicity & Durability

- Atomicity:
  - TXNs should either happen completely or not at all
  - If abort / crash during TXN, *no* effects should be seen
- Durability:
  - If DB stops running, changes due to completed TXNs should all persist
  - *Just store on stable disk*



We'll focus on how to accomplish atomicity (via logging)

Basic Idea: (Physical) Logging

The tables themselves has no notion of history, so we introduce a notion of history using a **log**

Idea:

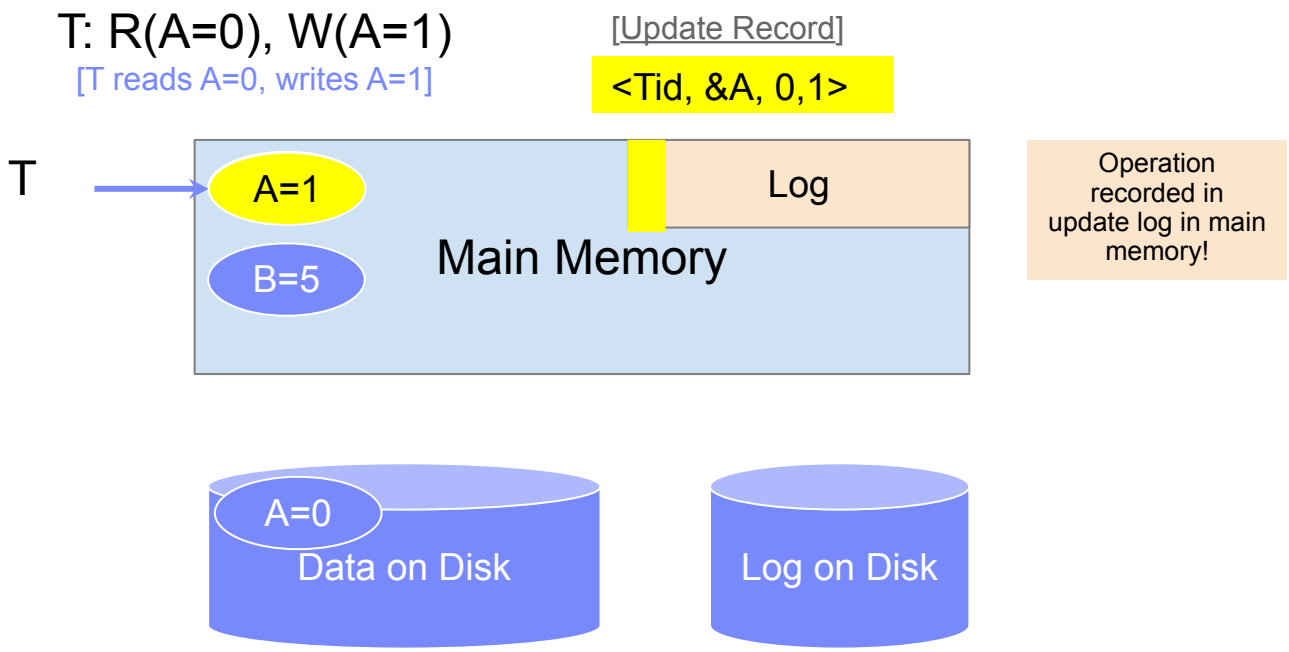
- Log consists of an ordered list of update records for ongoing transactions
- Log record contains UNDO information for every update!
  <TransactionID, location, old data, new data>

What DB does?

- Owns the log "service" for all applications/transactions
  - Logically (and usually physically) separate from the actual data
- Transparent to application or transaction
- Sequential writes to log, can **flush** — force writes to disk

This is sufficient to UNDO any transaction!

Using logging for atomicity



Why do we need logging for atomicity?

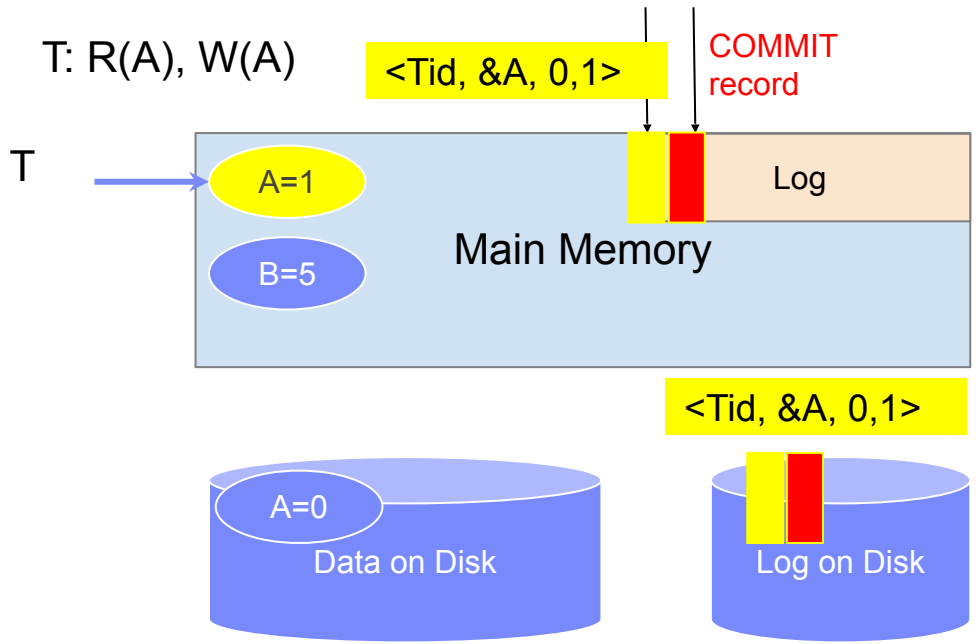
- Couldn't we just write TXN to disk **only** once whole TXN complete?
  - Then, if abort / crash and TXN not complete, it has no effect- atomicity!
  - *With unlimited memory and without performance constraints, this could work…*

- However, we **need write partial results of TXNs to disk** because of:
  - Memory constraints (enough space for full TXN??)
  - Time constraints (what if one TXN takes very long?)

We need to write partial results to disk!
…And so we need a **log** to be able to ***undo*** these partial results!

Write-ahead Logging (WAL) Commit Protocol

[Update Record]



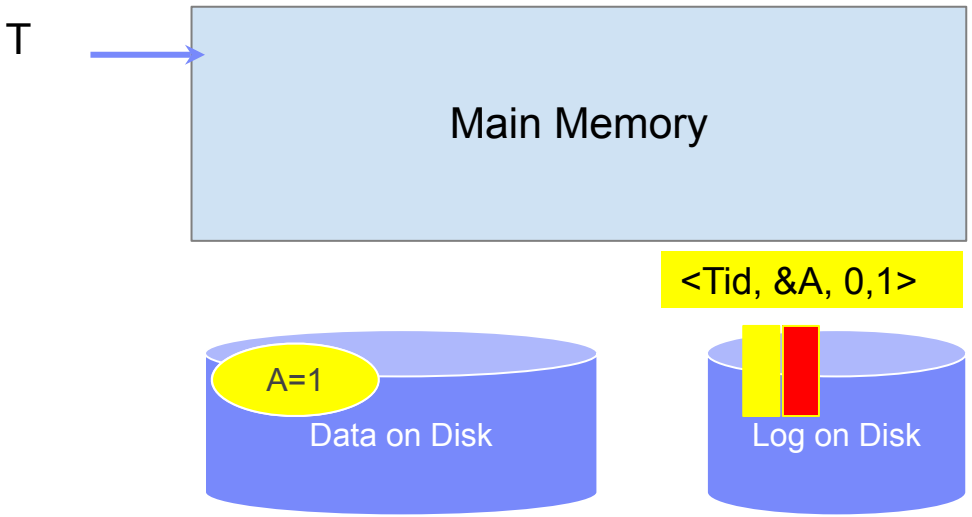
Commit after we've written log to disk but before we've written data to disk…

OK, Commit!

If we crash now, is T durable?

Write-ahead Logging (WAL) Commit Protocol

T: R(A), W(A)



Commit after we've written log to disk but before we've written data to disk… this is WAL!

OK, Commit!

If we crash now, is T durable?

USE THE LOG!

Write-Ahead Logging (WAL)

Algorithm: WAL

For each record update, write Update Record into LOG

Follow two Flush rules for LOG

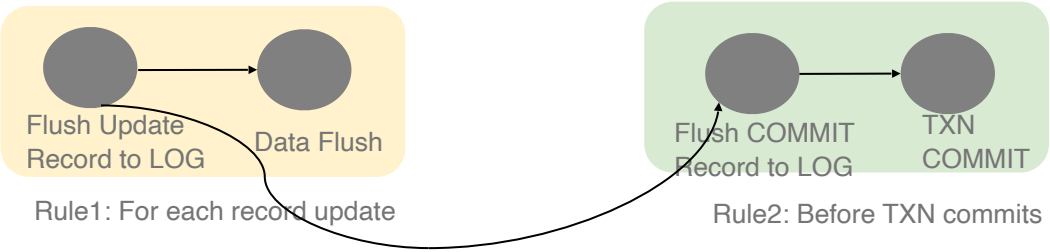
- **Rule1**: Flush Update Record *into LOG before* corresponding data page goes to storage
- **Rule2**: Before TXN commits,
  - Flush all Update Records to LOG
  - Flush COMMIT Record to LOG

Flush means write to disk

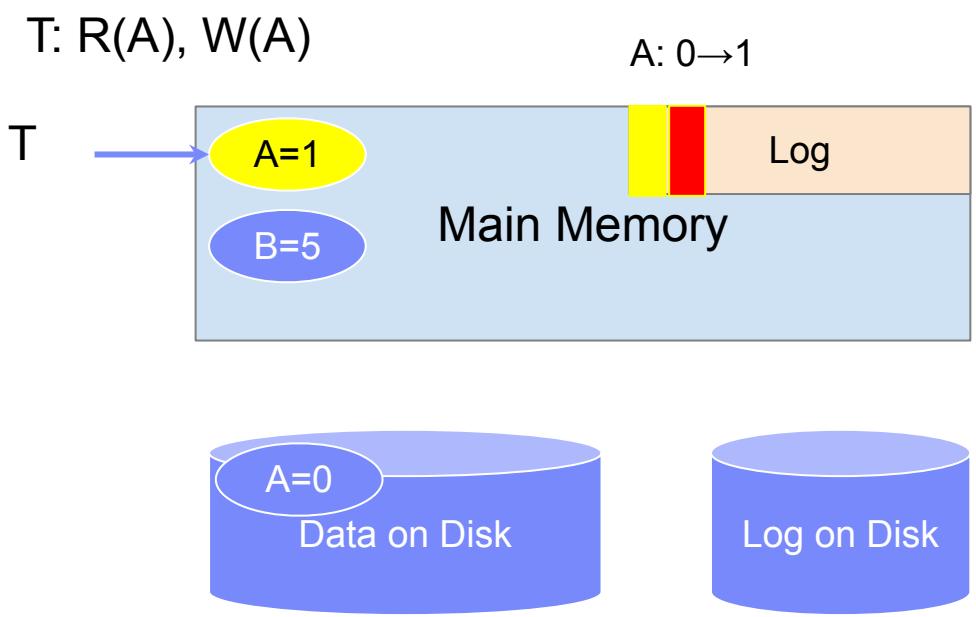
→ **Durability**

→ **Atomicity**

Transaction is committed *once COMMIT record is on stable storage*



Incorrect Commit Protocol #1



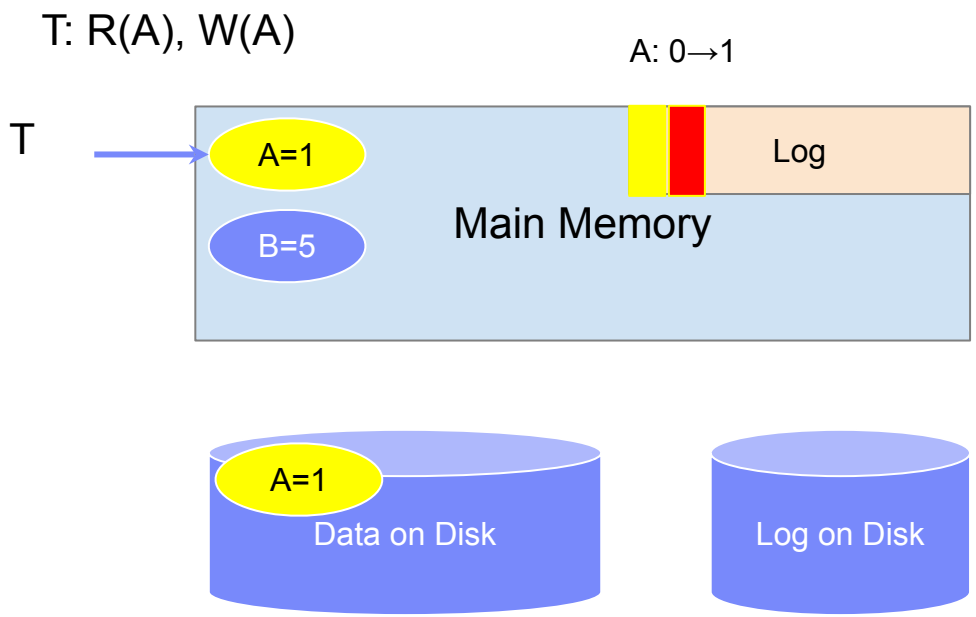
Let's try committing *before* we've written either data or log to disk…

OK, Commit!

If we crash now, is T durable?

Lost T's update!

Incorrect Commit Protocol #2



Let's try committing *after* we've written data but *before* we've written log to disk…

OK, Commit!

If we crash now, is T durable? Yes! Except…

How do we know whether T was committed??

Bank interest example: full run

Money

Account	....	Balance ($)
3001		500
4001		100
5001		20
6001		60
3002		80
4002		-200
5002		320
...		...
30108		-100
40008		100
50002		20

Money (@4:29 am day+1)

Account	....	Balance ($)
3001		550
4001		110
5001		22
6001		66
3002		88
4002		-220
5002		352
...		
30108		-110
40008		110
50002		22

WAL (@4:29 am day+1)

Update Records

Commit Record

T-Monthly-423	START TRANSACTION		
T-Monthly-423	3001	500	550
T-Monthly-423	4001	100	110
T-Monthly-423	5001	20	22
T-Monthly-423	6001	60	66
T-Monthly-423	3002	80	88
T-Monthly-423	4002	-200	-220
T-Monthly-423	5002	320	352
T-Monthly-423	...	...	...
T-Monthly-423	30108	-100	-110
T-Monthly-423	40008	100	110
T-Monthly-423	50002	20	22
T-Monthly-423	COMMIT		

‘T-Monthly-423’

Monthly Interest 10%
4:28 am Starts run on 10M bank accounts
Takes 24 hours to run

```
START TRANSACTION
        UPDATE Money
        SET Balance = Balance * 1.10
COMMIT
```

Bank interest example: with crash

Money

Account	....	Balance ($)
3001		500
4001		100
5001		20
6001		60
3002		80
4002		-200
5002		320
...		...
30108		-100
40008		100
50002		20

Money (@10:45 am)

Account	....	Balance ($)	
3001		550	??
4001		110	
5001		22	
6001		66	
3002		88	
4002		-200	??
5002		320	??
...			
30108		-110	
40008		110	
50002		22	??

WAL log (@10:29 am)

T-Monthly-423	START TRANSACTION		
T-Monthly-423	3001	500	550
T-Monthly-423	4001	100	110
T-Monthly-423	5001	20	22
T-Monthly-423	6001	60	66
T-Monthly-423	3002	80	88
T-Monthly-423	...	...	...
T-Monthly-423	30108	-100	-110
T-Monthly-423	40008	100	110
T-Monthly-423	50002	20	22
T-Monthly-423	4002	-200	-220
T-Monthly-423	5002	320	352

'T-Monthly-423'

Monthly Interest 10%
4:28 am Starts run on 10M bank accounts
Takes 24 hours to run
Network outage at 10:29 am,
System access at 10:45 am

Did T-Monthly-423 complete?
Which tuples are bad?

Case1: T-Monthly-423 was crashed
Case2: T-Monthly-423 completed. 4002 deposited 20$ at 10:45 am

Bank interest example: with recovery

Money (@10:45 am)

Account	....	Balance ($)
3001		550
4001		110
5001		22
6001		66
3002		88
4002		-200
5002		320
...		
30108		-110
40008		110
50002		22

Money (after recovery)

Account	....	Balance ($)
3001		500
4001		100
5001		20
6001		60
3002		80
4002		-200
5002		320
...		...
30108		-100
40008		100
50002		20

WAL log (@10:29 am)

T-Monthly-423	START TRANSACTION		
T-Monthly-423	3001	500	550
T-Monthly-423	4001	100	110
T-Monthly-423	5001	20	22
T-Monthly-423	6001	60	66
T-Monthly-423	3002	80	88
T-Monthly-423	...	...	...
T-Monthly-423	30108	-100	-110
T-Monthly-423	40008	100	110
T-Monthly-423	50002	20	22
[redacted]			

System recovery (after 10:45 am)

1. Undo uncommitted transactions
   - Restore old values from WALlog (if any)
   - Notify developers about aborted txn
1. Redo Recent transactions (w/ new values)
2. Back in business; Redo (any pending) transactions

A word on performance

- Question: why is a WAL good for performance?
  - Answer 1: updates to WAL are in sequential order
  - Answer 2: flushing the actual entries (i.e., the data in the tables) can be done lazily after the transaction was committed
    - Lets us have sequential writes also for the data, not just for the WAL!
- Sequential writes are very important both for flash and magnetic disk
  - In a couple of lectures we will understand why

An example of why sequential writes matter

Money

Account	....	Balance ($)
3001		500
4001		100
5001		20
6001		60
3002		80
4002		-200
5002		320
...		...
30108		-100
40008		100
50002		20

Money (@4:29 am day+1)

Account	....	Balance ($)
3001		550
4001		110
5001		22
6001		66
3002		88
4002		-220
5002		352
...		
30108		-110
40008		110
50002		22

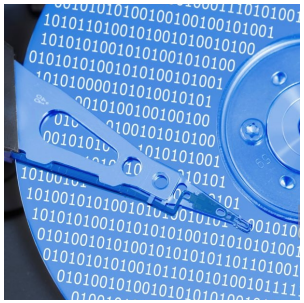
WAL (@4:29 am day+1)

T-Monthly-423	START TRANSACTION		
T-Monthly-423	3001	500	550
T-Monthly-423	4001	100	110
T-Monthly-423	5001	20	22
T-Monthly-423	6001	60	66
T-Monthly-423	3002	80	88
T-Monthly-423	4002	-200	-220
T-Monthly-423	5002	320	352
T-Monthly-423	...	...	...
T-Monthly-423	30108	-100	-110
T-Monthly-423	40008	100	110
T-Monthly-423	50002	20	22
T-Monthly-423	COMMIT		

<u>Cost to update all data</u>

10M bank accounts → 10M individual random writes? (worst case)

(@10 ms per write for magnetic disk, that's 100,000 secs)



<u>Speedup for commit</u>

100,000 secs vs 1 sec when written sequentially!!!

Summary so far

- If we follow the WAL rules, we get 2/4 of the ACID properties:
  - **A**tomicity
  - **D**urability
- We'll ignore consistency, since databases usually don't implement many constraints
- But what about **I**solation?
  - What happens if concurrent transactions interfere with each other?